Enterprise HR Policy & Employee Support Agentic RAG Copilot

FDE Project - Problem Statement, Proposed Solution, Deployment, and Easy-to-Understand Examples

Customer	Environment	Use Case
PeoplePrime Technologies (fictional)	5,000 employees	Internal HR Policy & Employee Support

Deployment Target
Deploy the complete production application on DigitalOcean Cloud using Docker services, including the FastAPI backend, UI, application dependencies, environment configuration, and production runtime.

1. Problem Statement

PeoplePrime Technologies has a large internal HR knowledge base containing leave policies, remote-work guidelines, attendance rules, payroll information, employee benefits, onboarding and offboarding procedures, code-of-conduct guidelines, employee-record policies, and HR forms. Although this information already exists, employees still contact HR repeatedly because they cannot quickly identify the correct document or understand which policy applies to their situation.

Traditional keyword search often returns multiple documents rather than one clear answer. A normal chatbot introduces another risk: it may confidently generate an answer even when reliable company evidence is unavailable. Some employee questions may also depend on current public information that has not yet been added to the private HR knowledge base.

The core business challenge
Employees need fast HR answers, while the company needs those answers to be grounded, trustworthy, transparent, and based on approved private company knowledge whenever possible.

2. Why Simple RAG Is Not Enough

Simple RAG	Problem in the real world
Question -> Retrieve -> Generate	Retrieved chunks may be weak, outdated, or unrelated, but the model may still generate an HR answer.
Uses only the private KB	It cannot answer well when required information is missing or when current public information is needed.
No evidence decision	The application does not decide whether the retrieved HR evidence is strong enough before answering.

3. Proposed Solution

Build an **Enterprise HR Policy & Employee Support Agentic RAG Copilot**. The system searches the trusted private HR knowledge base first, evaluates the retrieved evidence, and only generates a company-policy answer when the evidence is strong. If internal evidence is weak, LangGraph can route the request to Tavily web search. The web evidence is graded as well; if it remains insufficient, the system rewrites the query and retries rather than blindly answering.

1	Employee asks an HR-related question
2	LangGraph routes the request
3	Retrieve relevant chunks from the private Pinecone HR knowledge base
4	Grade whether the private evidence is strong enough
5A	GOOD -> Generate a grounded answer from company HR knowledge
5B	WEAK -> Search the web using Tavily
6	Grade the web evidence
7	If needed, rewrite the query and retry
8	Return the answer, sources, and decision trace

4. Easy Example #1 - Answer Found in the Company KB

Employee: "How many annual leave days do employees receive?"

The leave policy already exists in the private HR handbook. The system retrieves the relevant chunks from Pinecone, grades the evidence as useful, and generates the answer from internal company knowledge without unnecessary web search.

5. Easy Example #2 - Private Knowledge Is Not Enough

Employee: "What is the latest government public-holiday announcement for our location?"

The private HR documents may not contain the latest announcement. The evidence grader identifies the gap, LangGraph routes the request to Tavily, external evidence is evaluated, and the system responds only after obtaining sufficient evidence.

6. What Makes This Agentic RAG?

Unlike a fixed Question -> Retrieve -> Generate pipeline, this system makes decisions during execution: retrieve, grade, route, search, rewrite, retry, and generate only when evidence is sufficient.

Normal RAG	Agentic RAG in this project
Retrieve once	Retrieve, grade, and retry when necessary
Fixed flow	Conditional LangGraph routing
Generate after retrieval	Generate only after evidence evaluation
Private KB only	Private HR KB first, controlled web fallback when needed

7. Proposed Product Components

• **LangGraph:** agentic decision workflow.

• **Pinecone:** private embedded HR knowledge.

• **HuggingFace embeddings:** vector representations for documents and queries.

• **Groq LLM:** routing, grading, query rewriting, and grounded generation.

• **Tavily:** controlled web fallback when private evidence is insufficient.

• **FastAPI:** production backend APIs.

• **HTML/CSS/JavaScript:** employee-facing HR assistant interface and workflow trace.

• **SQLite:** audit logging for debugging and transparency.

• **Document ingestion:** authorized HR staff can add PDF, TXT, Markdown, and DOCX knowledge.

• **Docker:** packages the application and its runtime consistently for deployment.

• **DigitalOcean Cloud:** hosts the containerized production application.

8. Expected Business Value

• Reduce repetitive HR support questions.

• Help employees find and understand policies faster.

• Prioritize approved private HR knowledge.

• Reduce unsupported answers through evidence grading.

• Handle knowledge gaps with controlled web fallback.

• Provide source and workflow traces for transparency.

• Create a deployable, production-style HR AI product rather than only a notebook demo.

9. Deployment - DigitalOcean + Docker

The final product will be containerized with Docker and deployed to DigitalOcean Cloud. The deployment demonstrates the Forward Deployed Engineer lifecycle beyond local development: package the FastAPI application and frontend, configure production environment variables and external services, run the container in the cloud, expose the application to users, validate the deployed workflow, and observe the running product.

1	Build and test the complete application locally
2	Create the Dockerfile and container image
3	Configure required environment variables and API credentials
4	Provision the DigitalOcean cloud runtime
5	Deploy and run the Dockerized FastAPI + UI application
6	Connect production services such as Pinecone, Groq, and Tavily
7	Verify ingestion, Agentic RAG workflow, sources, and audit trace in production
8	Observe, troubleshoot, and improve the deployed product

10. FDE Perspective

The goal of the Forward Deployed Engineer is not simply to demonstrate RAG in a notebook. The FDE starts from the customer's HR support problem and delivers a usable product: knowledge ingestion, agentic orchestration, APIs, UI, auditability, testing, Docker packaging, cloud deployment, observation, and continuous improvement.

Customer Problem -> Understand Workflow -> Connect HR Knowledge -> Build Agentic RAG -> Expose APIs -> Build UI -> Test -> Dockerize -> Deploy on DigitalOcean -> Observe -> Improve